

자바(Java) 소개

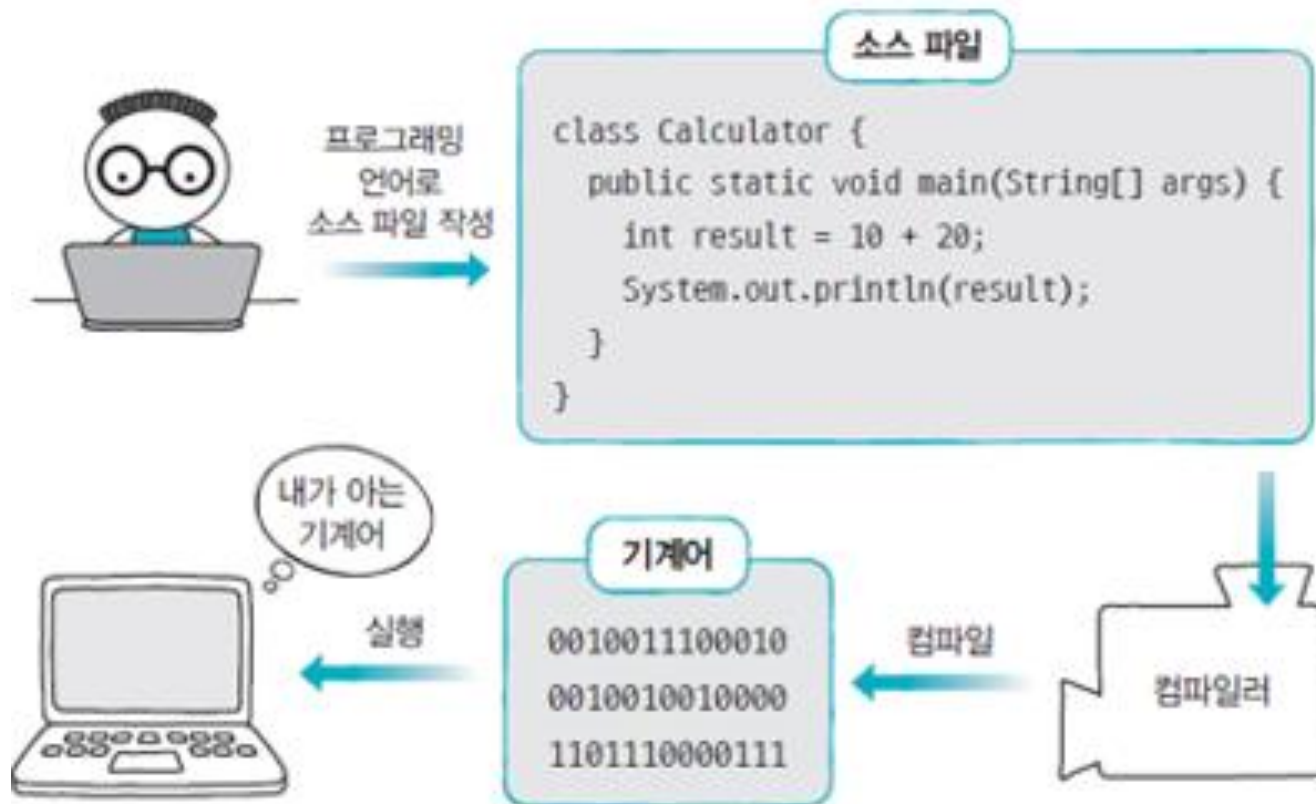
목차

- 프로그램 언어란?
- JAVA 언어 소개
- 가상머신과 플랫폼
- 개발환경 구성하기
- JAVA 프로그램의 구조
- JAVA의 기본 데이터 타입

프로그래밍 언어

□ 프로그래밍 언어의 역할

- ▣ 사람과 컴퓨터 사이의 대화를 가능하게 하는 역할



프로그래밍 언어

□ 프로그램 작성 언어

▣ 기계어(machine language)

- 0, 1의 이진수로 구성된 언어
- 컴퓨터의 CPU는 기계어만 이해하고 처리가능

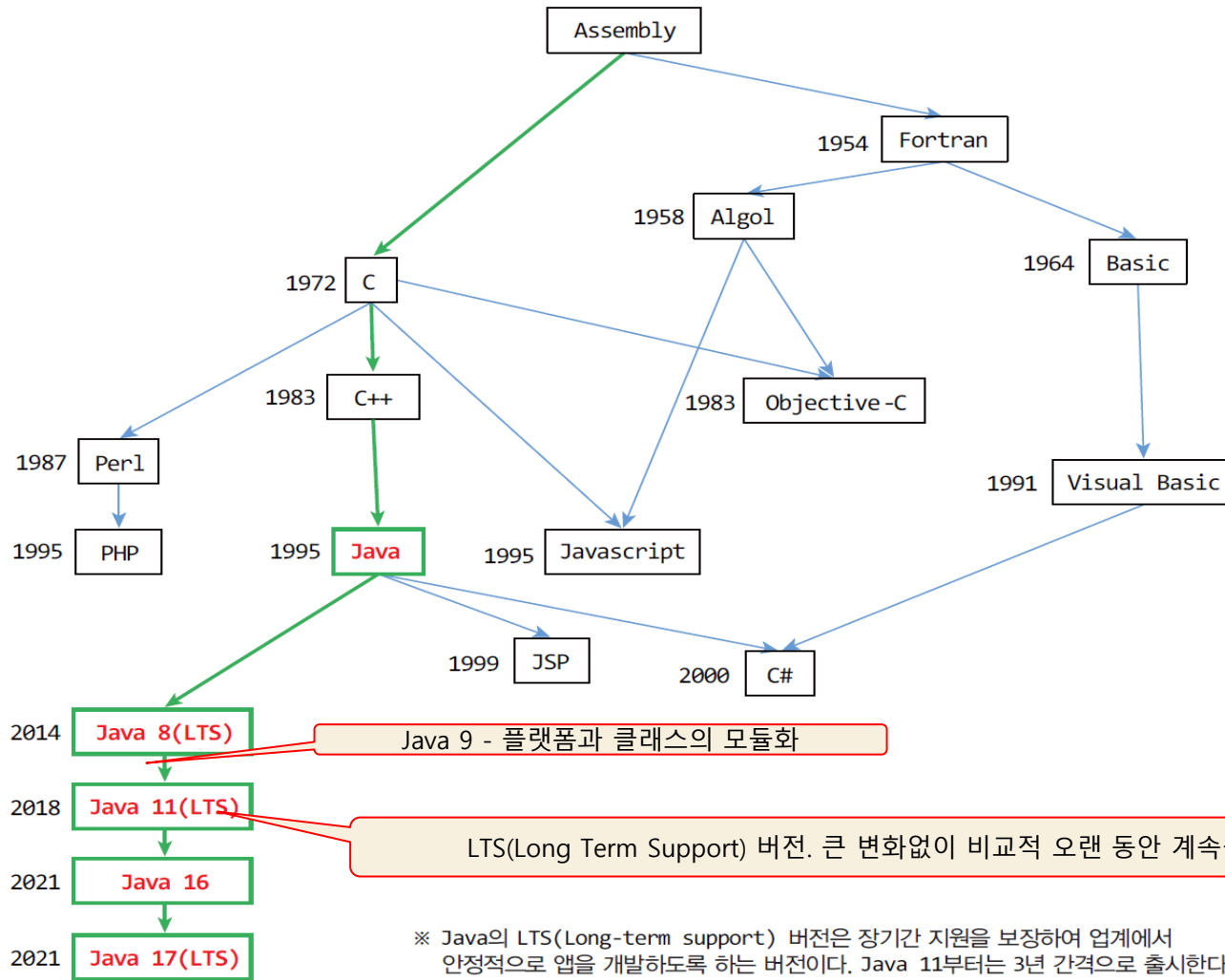
▣ 어셈블리어

- 기계어 명령을 ADD, SUB, MOVE 등과 같은 표현하기 쉬운 상징적인 단어인 니모닉 기호 (mnemonic symbol)로 일대일 대응시킨 언어

▣ 고급언어

- 사람이 이해하기 쉽고, 복잡한 작업, 자료 구조,알고리즘을 표현하기 위해 고안된 언어
 - 자바(Java) : 임베디드, 웹, 안드로이드 앱 개발 분야
 - C/C++ : 하드웨어(HW) 장치 제어 및 임베디드 SW 분야
 - C++/C# : 데스크탑 앱 또는 ASP.NET 기반 웹 SW 분야
 - Python : 빅데이터 분석 및 머신러닝 SW 분야

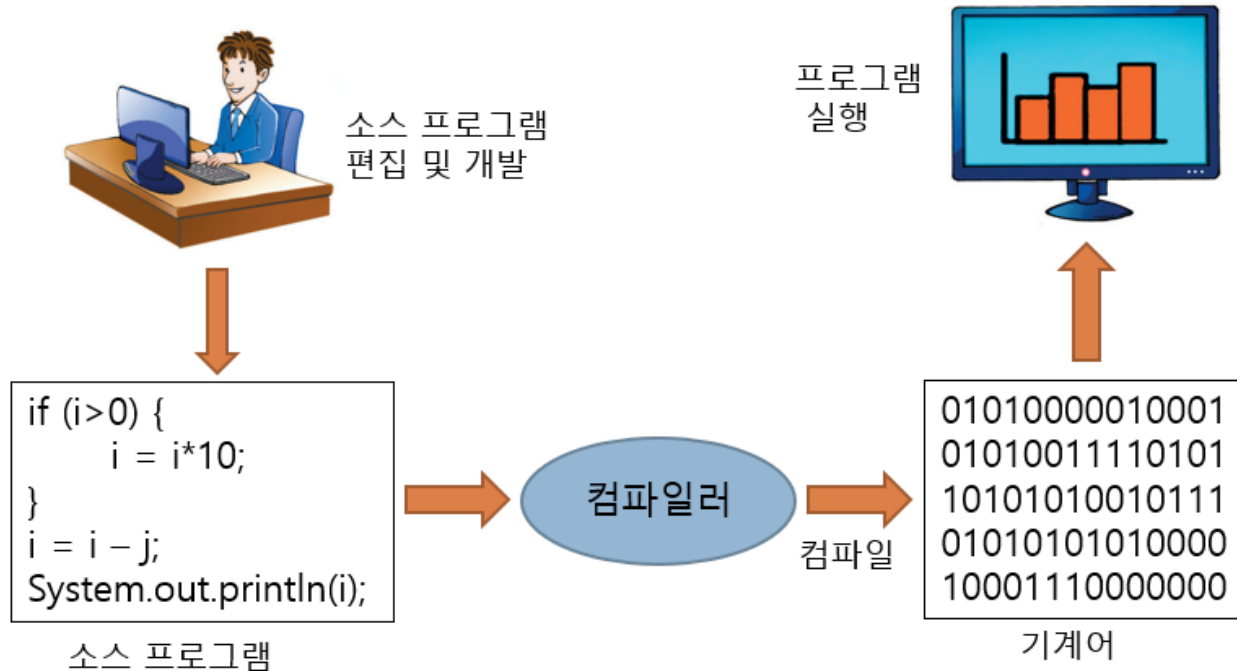
프로그래밍 언어의 진화



※ Java의 LTS(Long-term support) 버전은 장기간 지원을 보장하여 업계에서 안정적으로 앱을 개발하도록 하는 버전이다. Java 11부터는 3년 간격으로 출시한다.

프로그래밍과 컴파일

- 소스 : 프로그래밍 언어로 작성된 텍스트 파일
- 컴파일 : 소스 파일을 컴퓨터가 이해할 수 있는 기계어로 만드는 과정
 - 자바 : .java -> .class
 - C : .c -> .obj-> .exe
 - C++ : .cpp -> .obj -> .exe



Java 언어의 개요

□ Java의 탄생

- 썬 마이크로시스템즈의 제임스 고슬링(James Gosling)에 의해 시작
 - 1991년 냉장고, TV 등의 가전 제품에 들어갈 소프트웨어를 위해 개발
- 처음에는 오크(Oak)라고 불리다가, 1995년 Sun World에서 Java로 공식 발표
- 2009년에 선마이크로시스템즈를 오라클이 인수
 - 오라클이 자바 개발 도구 배포



□ 특징

- 모든 운영체제에서 실행가능
- 객체 지향 프로그래밍(OOP : Object-Oriented Programming)
 - C/C++의 포인터, 연산자 오버로딩, 다중상속 등 제거
- 메모리 자동 정리
- 풍부한 무료 라이브러리

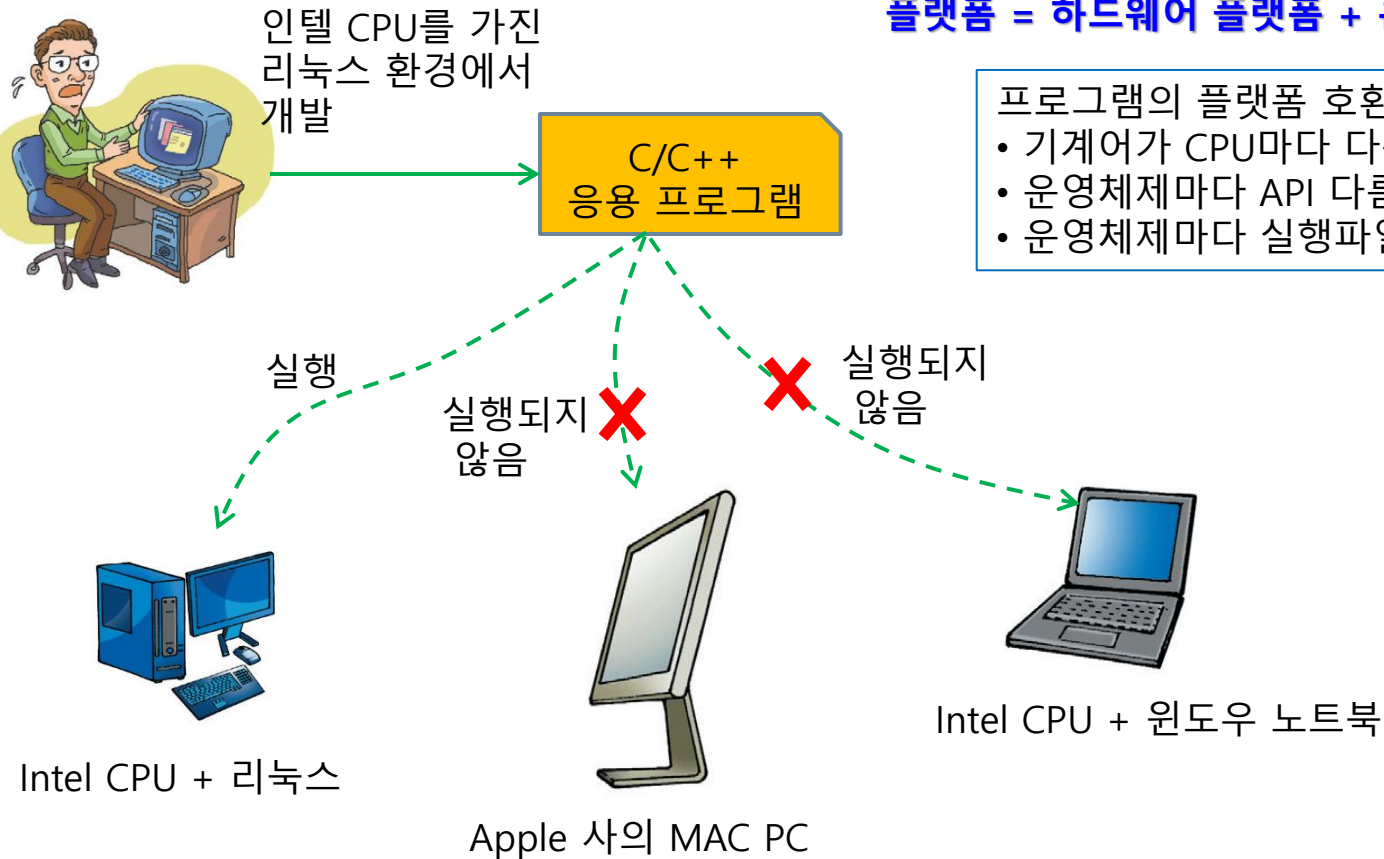


마스코트
듀크



로고
커피컵

기존 언어의 플랫폼 종속성 (Platform Dependency)



플랫폼 = 하드웨어 플랫폼 + 운영체제 플랫폼

프로그램의 플랫폼 호환성 없는 이유

- 기계어가 CPU마다 다름
- 운영체제마다 API 다름
- 운영체제마다 실행파일 형식 다름

자바의 플랫폼 독립성, WORA

□ WORA(Write Once Run Anywhere)

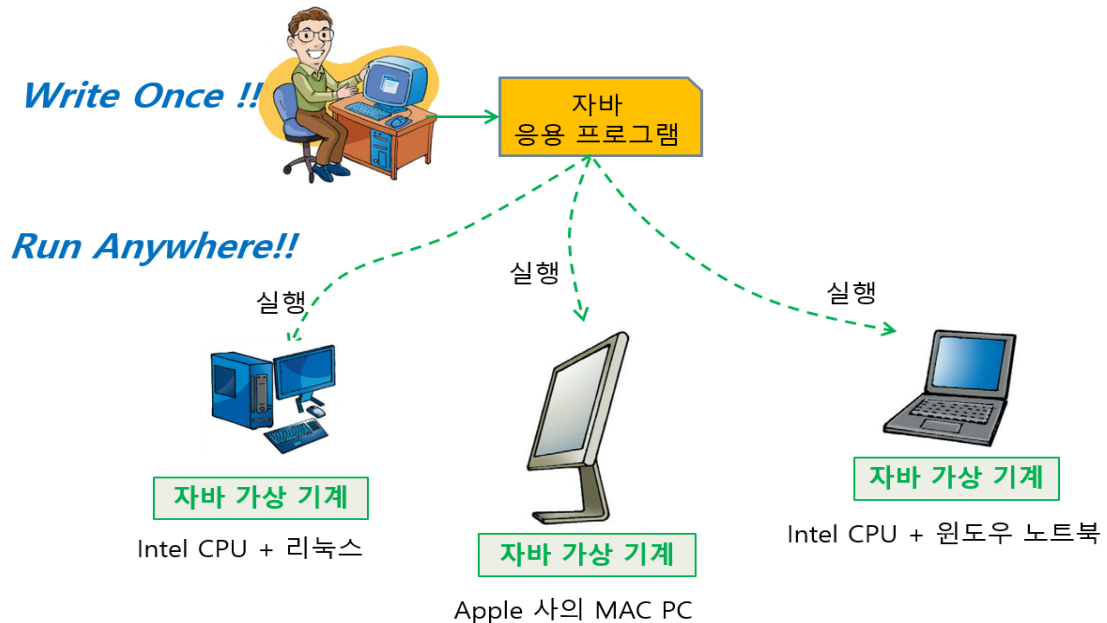
- 한번 작성된 코드는 모든 플랫폼에서 바로 실행되는 자바의 특징
- C/C++ 등 기존 언어가 가진 플랫폼 종속성 극복
 - OS, H/W에 상관없이 자바 프로그램이 동일하게 실행

□ WORA를 가능하게 하는 자바의 특징

□ JVM(Java Virtual Machine)

□ 바이트 코드(byte code)

- JVM에서 실행가능한
바이너리 코드



자바 가상 기계와 자바 실행 환경

□ 바이트 코드

- 클래스 파일(.class)에 저장
- 자바 가상 기계에서 실행 가능한 바이너리 코드
 - 바이트 코드는 컴퓨터 CPU에 의해 직접 실행되지 않음
 - 자바 가상 기계가 작동 중인 플랫폼에서 실행
 - 자바 가상 기계가 인터프리터 방식으로 바이트 코드 해석

□ 자바 가상 기계(JVM : Java Virtual Machine)

- 각기 다른 플랫폼에 설치
- 동일한 자바 실행 환경 제공
- 자바 가상 기계 자체는 플랫폼에 종속적
 - 자바 가상 기계는 플랫폼마다 각각 작성됨
 - 예) 리눅스에서 작동하는 자바 가상 기계는 윈도우에서 작동하지 않음
- 자바 가상 기계 개발 및 공급
 - 자바 개발사인 오라클 외 IBM, MS 등 다양한 회사에서 제작 공급

□ 자바의 실행

- 자바 가상 기계가 클래스 파일(.class)의 바이트 코드 실행

자바 응용프로그램의 실행

* 자바는 링크 과정 없음



자바 프로그래밍

Draw.java

Hello.java

Shape.java

(소스 코드)

자바 컴파일러

javac.exe

Draw.class

Hello.class

Shape.class

(바이트 코드)

java.exe

Object.class

String.class

클래스 로딩

자바 가상 기계

윈도우 운영체제

인텔 PC H/W

자바 가상 기계

Apple 맥 운영체제

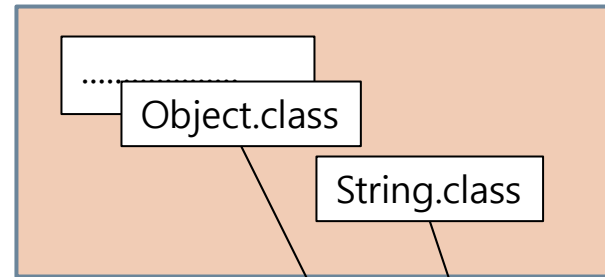
Apple H/W

자바 가상 기계

휴대폰 운영체제

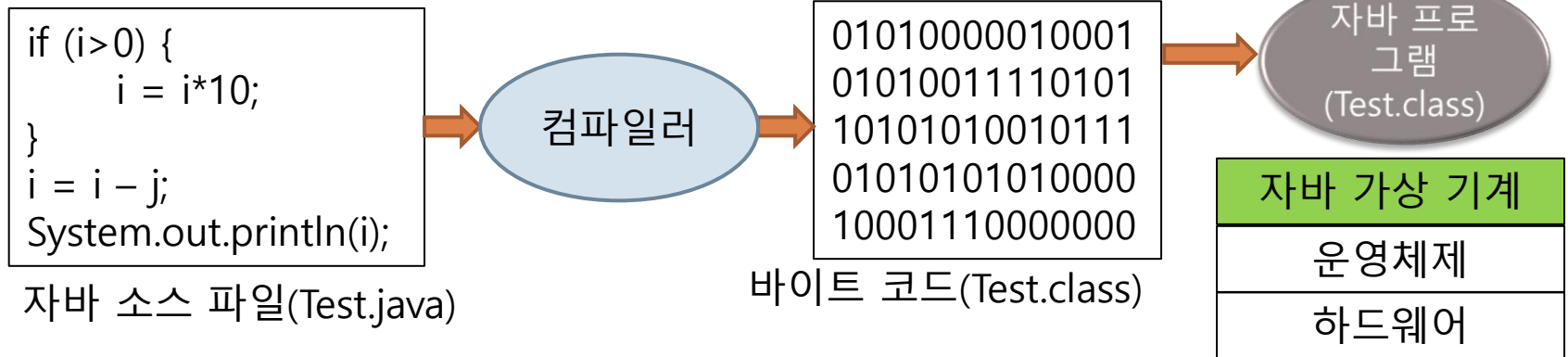
휴대폰 H/W

실행에 필요한 자바 클래스 라이브러리(JDK APIs)

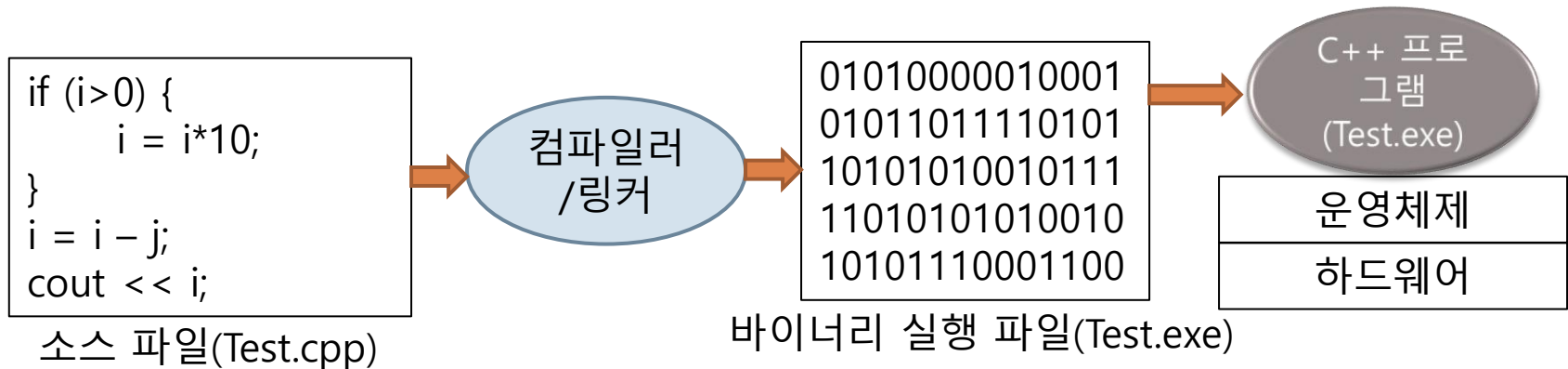


자바와 타언어(C/C++)의 실행 차이

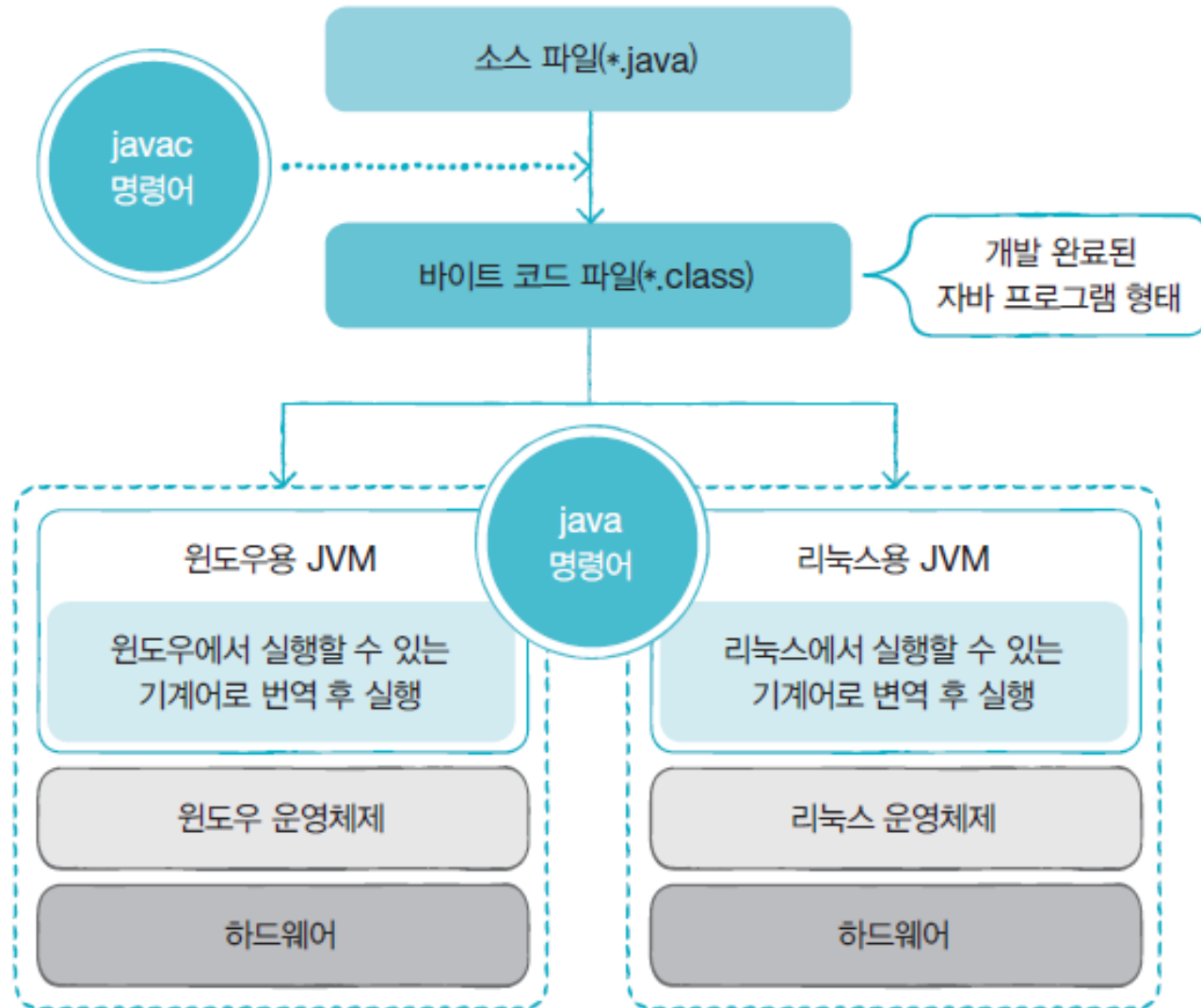
□ 자바



□ C/C++



자바 응용프로그램의 실행 단계



자바 배포판

- 개발 환경에 따라 다양한 자바 개발 배포판 제공

- **Java SE (Standard Edition)**

- ▣ 데스크탑과 서버 응용 개발 플랫폼

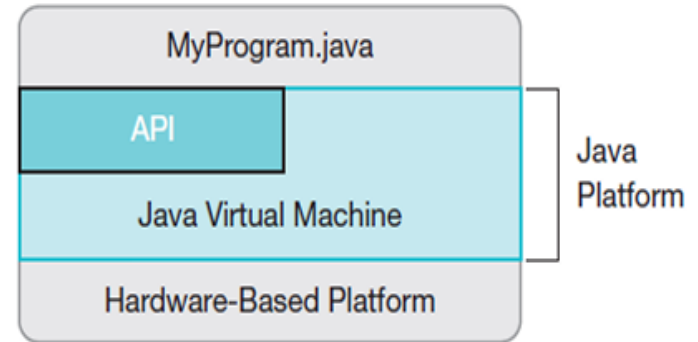
- **Java ME (Micro Edition)**

- ▣ 휴대 전화나 PDA, 셋톱박스 등 제한된 리소스를 갖는 하드웨어에서 응용 개발을 위한 플랫폼

- ▣ Java SE의 서브셋 + 임베디드 및 가전 제품을 위한 API 정의

- **Java EE (Enterprise Edition)**

- ▣ 자바를 이용한 다중 사용자, 기업용 응용 개발을 위한 플랫폼
 - ▣ Java SE + 인터넷 기반의 서버사이드 컴퓨팅 관련 API 추가



자바와 오픈 소스

□ Oracle JDK (Java Development Kit) 유료화

- ▣ Java 자체는 무료이지만, JDK는 무료가 아님
- ▣ Java 유료화 정책 (Java SE subscription)
 - Oracle JDK 11 이상 유료
 - 개인 사용자에게만 무료, 상업적 이용 시 이용료 지불
 - Oracle JDK 17 부터 무료 (Oracle NFTC : Oracle No-Fee Terms and Conditions)
 - 학습 및 교육 목적의 사용은 무료
 - <https://www.oracle.com/downloads/licenses/no-fee-license.html>

□ Open JDK

- ▣ 오픈 소스 기반의 OpenJDK
 - Oracle JDK도 OpenJDK 소스코드를 기반으로 제작
- ▣ 다운로드 : <https://openjdk.org/>

자바 개발 환경 구성하기

JDK와 JRE

□ JDK(Java Development Kit)

- ▣ 자바 응용 개발 환경
- ▣ 개발에 필요한 도구 포함
 - 컴파일러, JRE (Java Runtime Environment), 다양한 API 및 샘플 등 포함

□ JRE(Java Runtime Environment)

- ▣ 자바 실행 환경, JVM 포함
- ▣ Class library

□ JDK의 bin 디렉터리에 포함된 주요 개발 도구

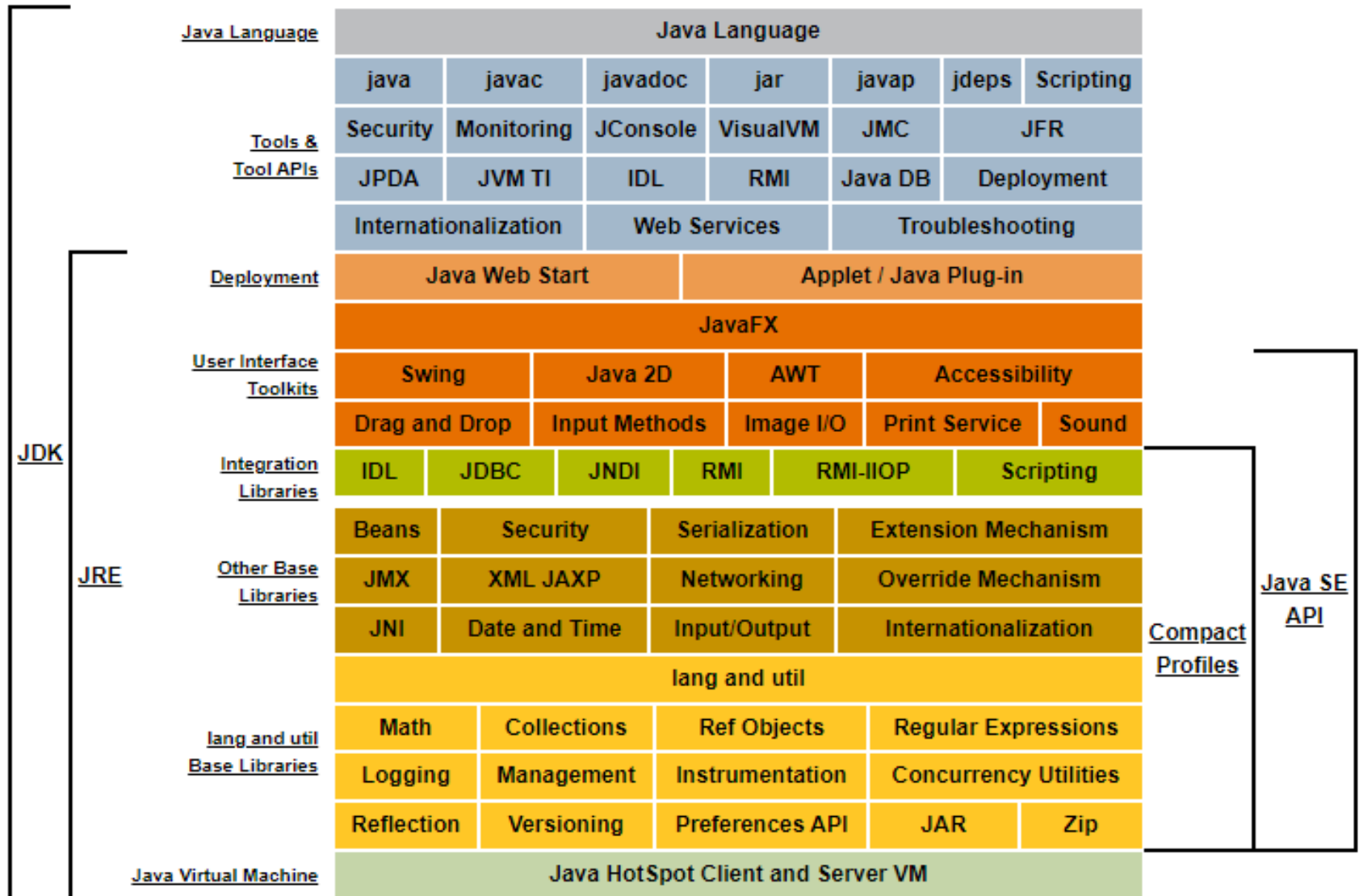
- ▣ javac - 자바 소스를 바이트 코드로 변환하는 컴파일러
- ▣ java - jre의 bin 디렉터리에 있는 자바 응용프로그램 실행기
- ▣ jar - 자바 아카이브 파일 (JAR)을 생성 및 관리하는 유틸리티

자바 API

- **자바 API(Application Programming Interface)**
 - ▣ 개발자가 이용하여 쉽고 빠르게 자바 프로그램을 개발할 수 있는 자바 라이브러리
 - 클래스 라이브러리로 제공
 - 패키지 형태로 제공
- **자바 패키지(package)**
 - ▣ 서로 관련된 클래스들을 분류하여 묶어 놓은 것
 - ▣ 계층구조로 되어 있음
 - ▣ 개발자 자신의 패키지 생성 가능

JDK와 JRE

Description of Java Conceptual Diagram



자바 통합 개발 환경

- **IDE(Integrated Development Environment)란?**
 - ▣ 통합 개발 환경
 - ▣ 편집, 컴파일, 디버깅을 한번에 할 수 있는 통합된 개발 환경
- **이클립스(Eclipse)**
 - ▣ 자바 응용 프로그램 개발을 위한 통합 개발 환경
 - ▣ IBM에 의해 개발된 오픈 소스 프로젝트
 - ▣ <https://www.eclipse.org/downloads/> 에서 다운로드
- **Visual Studio Code**
 - ▣ 마이크로소프트에서 개발하여 2016년 출시
 - ▣ <https://code.visualstudio.com/> 에서 다운로드
- **IntelliJ IDEA**
 - ▣ JetBrains사의 통합 개발환경
 - ▣ <https://www.jetbrains.com/>에서 다운로드

IntelliJ IDEA 설치

IntelliJ IDEA 소개

□ **JetBrains에서 개발한 통합 개발환경 (IDE)**

□ **주요 기능**

- **스마트 코드 완성**

- 문맥을 이해하고 적절한 코드 완성을 제공

- **코드 분석 및 리팩토링**

- 실시간 코드 검사와 안전한 리팩토링 도구 지원

- **멀티 언어 지원**

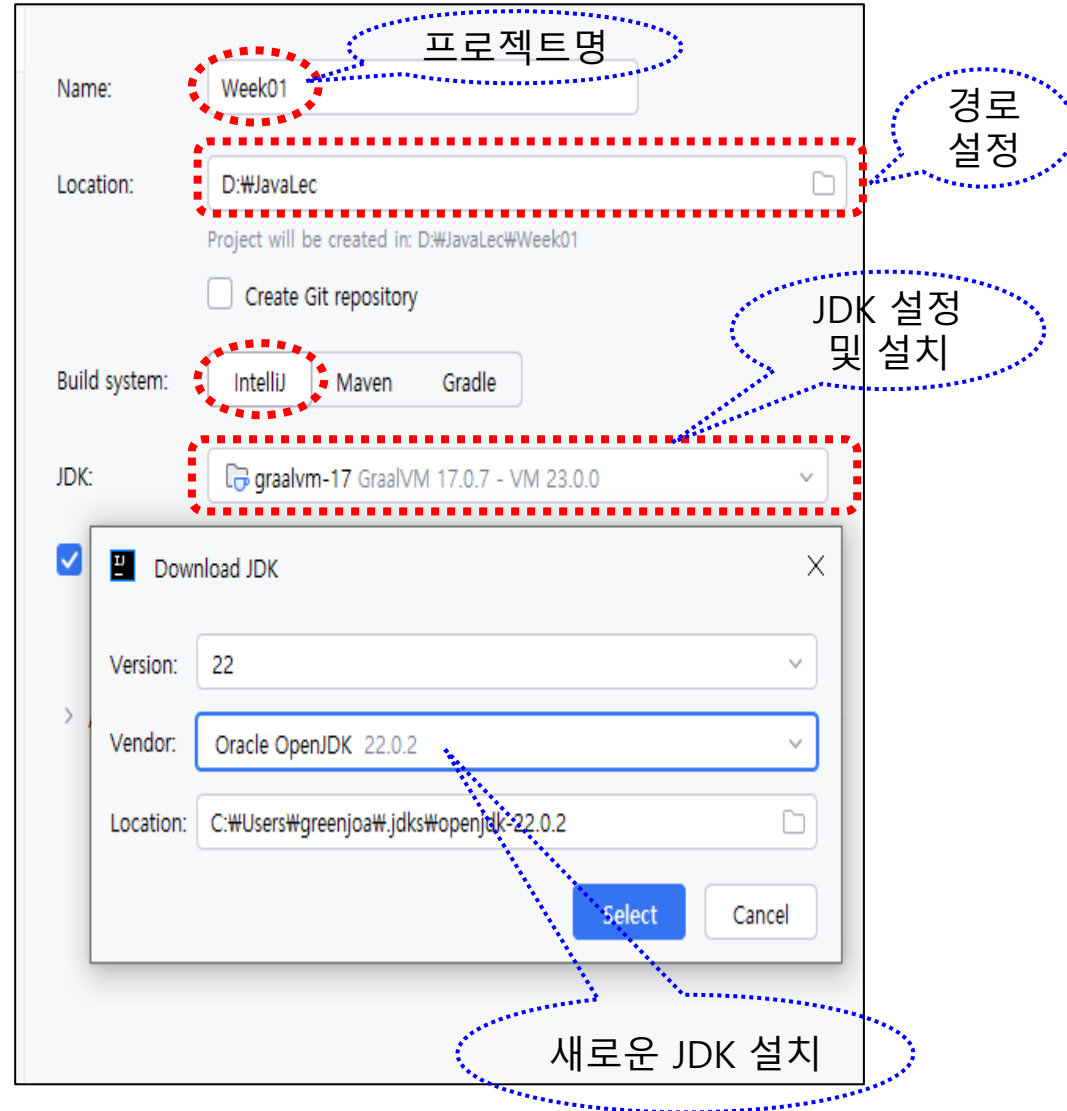
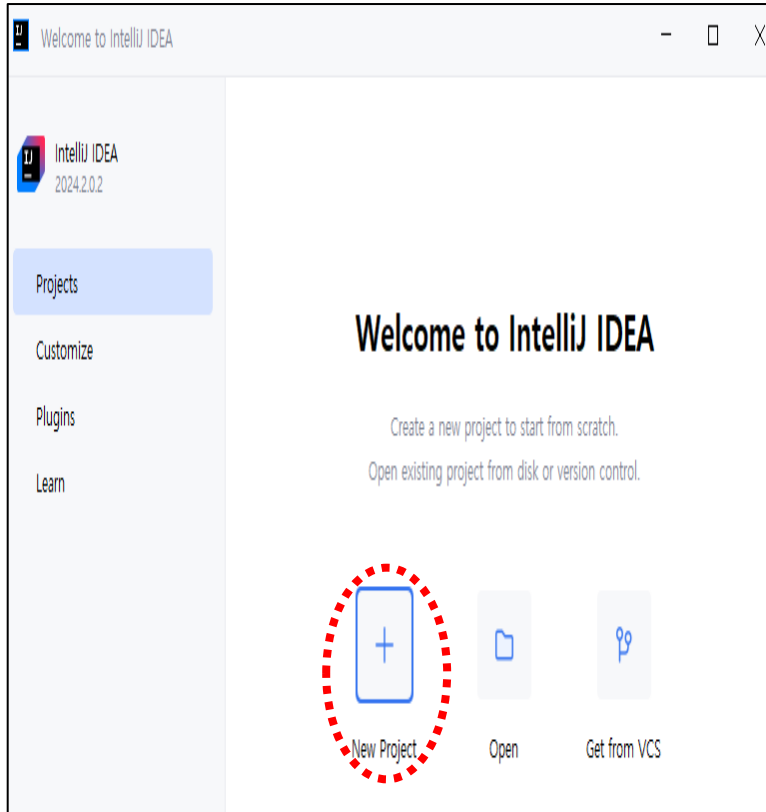
- Java, Kotlin, Groovy, Scala 등 다양한 언어 지원

- **디버깅 및 테스트**

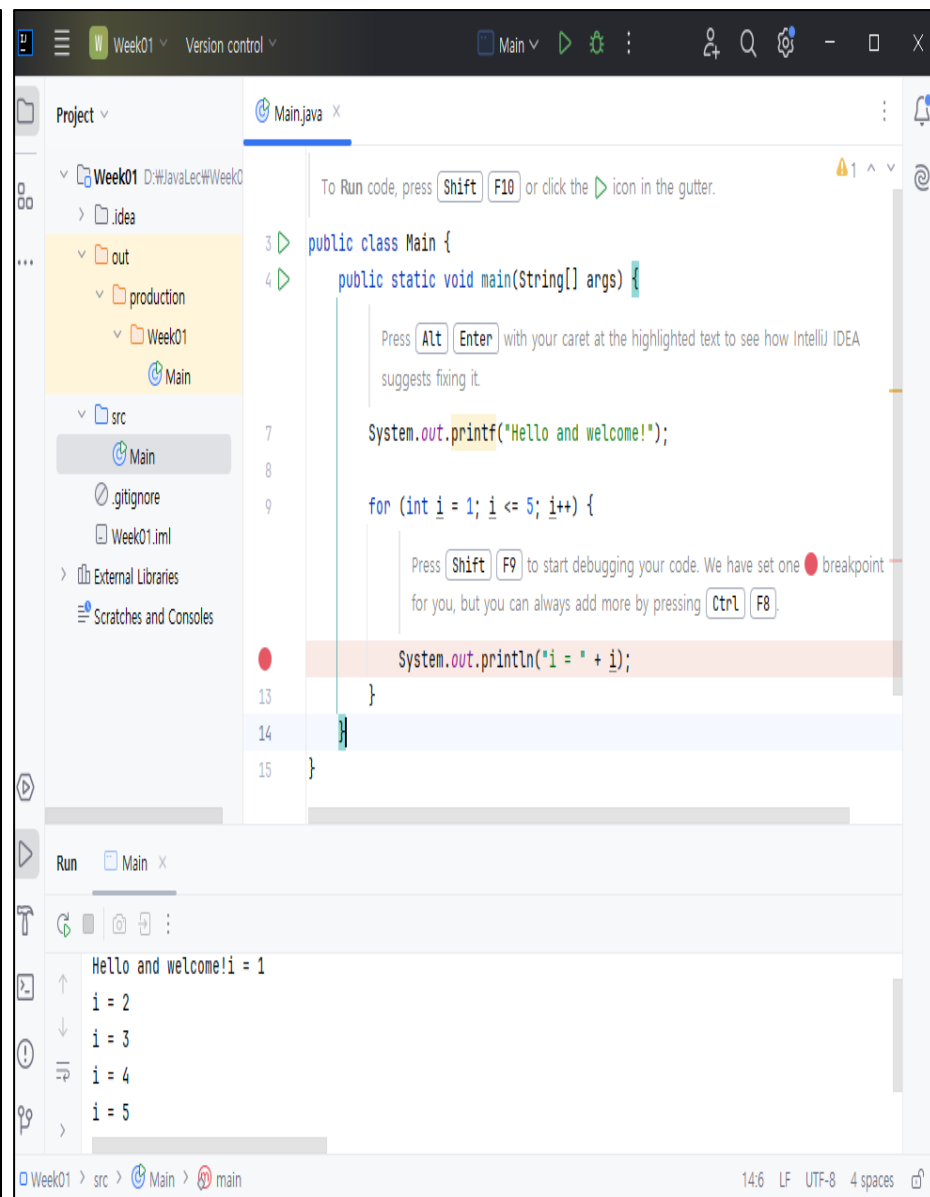
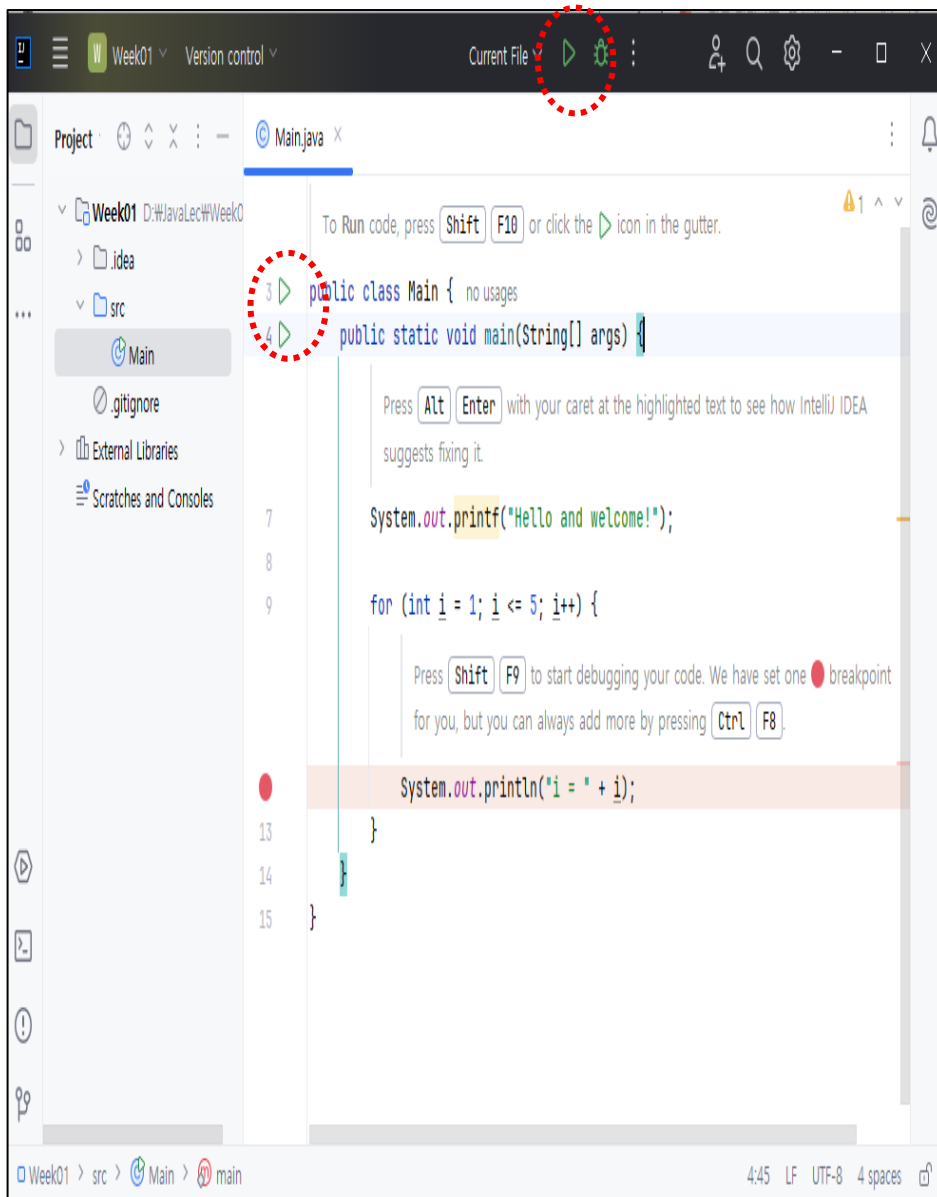
- 강력한 디버깅 기능과 테스트 도구 지원

□ **설치 방법은 자료실 자료 참고할 것**

프로젝트 생성



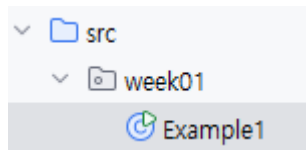
프로젝트 생성



실습 예제 – Java Class 만들기

□ 실습 내용: 프로젝트 생성후 패키지과 소스 파일 만들기

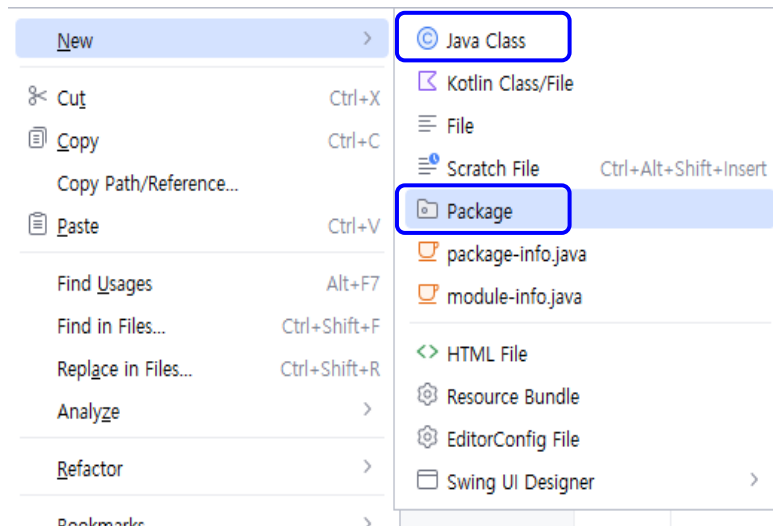
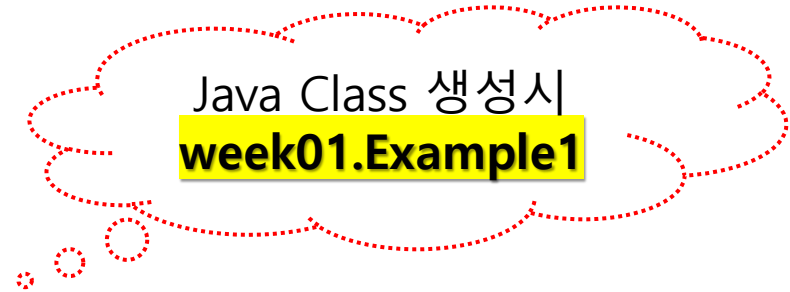
■ 프로젝트명 : JavaLec



■ 패키지 생성 : week01

■ 패키지 선택 후 Java 클래스 생성 : Example1

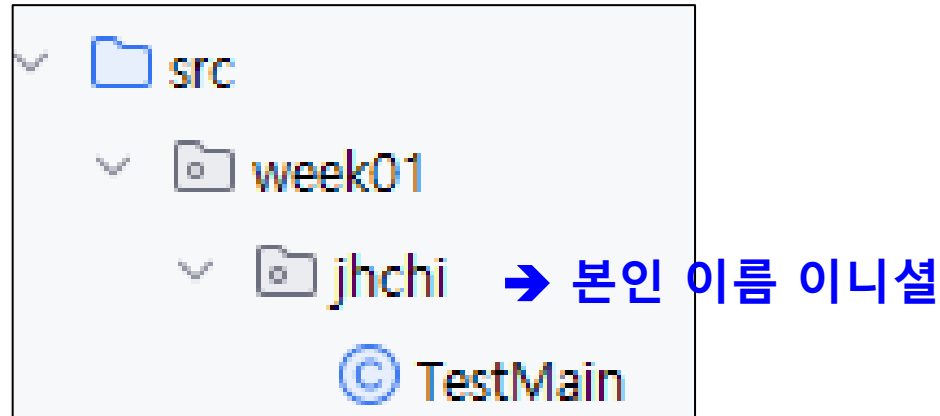
패키지와 Java Class파일 한번에 생성



실습예제 – 패키지와 Class 만들기

□ 실습내용

- ▣ 다음의 구조로 Java Class 파일 (TestMain) 만들기



Java 프로그램의 기본구조

Hello, 자바 프로그램의 기본 구조

```
/*
 * 소스 파일 : Hello.java
 */
// public 클래스는 한 파일에 하나! Public클래스의 이름은 파일이름과 똑같이!!!
public class Hello {

    public static int sum(int n, int m) {
        return n + m;
    }

    // main() 메소드에서 실행 시작
    static void main(String[] args) {
        int i = 20;
        int s;
        char a;

        s = sum(i, 10); // sum() 메소드 호출
        a = '?';
        System.out.println(a); // 문자 '?' 화면 출력
        System.out.println("Hello"); // "Hello" 문자열 화면 출력
        System.out.println(s); // 정수 s 값 화면 출력
    }
}
```

클래스

메소드

메소드

<실행결과>

```
?
Hello
30
```

예제 설명

□ 클래스 만들기

- Hello 이름의 클래스 선언

```
public class Hello {  
}
```

- class 키워드로 클래스 선언
- public으로 선언하면 다른 클래스에서 접근 가능
- 클래스 코드는 {} 내에 모두 작성

□ 주석문

- // 한 라인 주석
- /* 여러 행 주석 */
- /** 문서화 주석 */
 - javadoc -d doc hello.java -encoding utf-8

□ main() 메소드

- 자바 프로그램은 main()에서 실행 시작

```
static void main(String[] args) {  
}
```

- public static void으로 선언
- String[] args로 실행 인자를 전달 받음
- **static void main() / void main(){ }**

→ java 21이후 부터 가능 → 불필요한 보일러플레이트 코드 줄이기

□ 메소드

- C/C++에서의 함수를 메소드로 지칭

```
public static int sum(int n, int m) {  
    ...  
}
```

- **클래스 바깥에 작성할 수 없음**

□ 메소드 호출

- sum() 메소드 호출

```
s = sum(i, 10);
```

- sum() 호출 시 변수 i의 값과 정수 10을 전달
- sum()의 n, m에 각각 20, 10 값 전달
- sum()은 n과 m 값을 더한 30 리턴
- 변수 s는 정수 30을 전달받음

예제 설명 (계속)

□ 변수 선언

□ 변수 타입과 변수 이름 선언

```
int i=20;  
char a;
```

□ 메소드 내에서 선언된 변수는 지역 변수

- 지역 변수는 메소드 실행이 끝나면 저장 공간 반환

□ 문장

□ ;로 한 문장의 끝을 인식

```
int i=20;  
s = sum(i, 20);
```

□ 화면 출력

□ 표준 출력 스트림에 메시지 출력

```
System.out.println("Hello"); // "Hello" 화면 출력
```

□ 표준 출력 스트림 System.out의 println() 메소드 호출

- println()은 여러 타입의 데이터 출력 가능
- println()은 출력 후 다음 행으로 커서 이동

□ Java 23부터 간소화된 입출력 기능 지원

- System.out을 생략

```
println("Hello"); // "Hello" 화면 출력
```

실습예제 - 오류 수정하기

□ 실습내용

- ▣ 패키지 만들기 : week01.example01
- ▣ 배포한 Errors.java 파일 복사해서 생성한 패키지에 붙여넣기
- ▣ 오류를 수정 및 실행한 후 출력 결과를 확인

본인이름
이니셜

```
public class errors
{
    public Static main(string[] args) {
        System.out.println("이 프로그램은 많은 오류를 가지고 있다.");
        System.out.println("그러나 프로그램이 이 문장을 출력된다면")
        System.out.PrinTln("모든 오류를 고친 것이다.");
    }
}
```

<출력 결과>

이 프로그램은 많은 오류를 가지고 있다.
그러나 프로그램이 이 문장을 출력된다면
모든 오류를 고친 것이다.

실습 예제 – 문서화 주석 추가하기

□ 실습내용 – 클래스 및 메서드에 문서화 주석 구문 추가하기

▣ 클래스에 대한 주석 추가

```
/**
 * 이 클래스는 자바 프로그램의 구조를 파악하기 위한 오류를 수정하기 위한 클래스 입니다.
 * @author greenjoa
 * @version 1.0
 * @since 09-01
 */
public class Errors
```

▣ 메서드에 대한 주석 추가

```
/**
 * 프로그램 시작을 위한 메인 함수입니다.
 * @param args 메인 함수 인자
 */
static void main(String[] args) {
```

실습 예제 - 문서화 주석 추가하기

- Tools 메뉴 >> JavaDoc 생성 (Generate Java Doc)

생성된 문서자 저장될 디렉토리 지정

문서에 포함될 접근 제어자 수준 (private-모두 렌더링-전체 구조 파악할 때 사용)

한글 깨짐 방지 설정

데이터 타입

데이터 타입

□ 데이터 타입

- ▣ 자료에 대한 형태를 지정하는 것
- ▣ 데이터 타입의 역할
 - 데이터가 가질 자료형을 지정
 - 데이터가 가질 메모리 크기를 지정

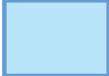
□ 데이터 타입 종류

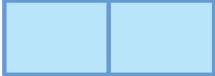
- ▣ 기본형 데이터 타입 (Primitive Data Type)
 - JVM이 미리 정해놓은 기본적인 타입 (8가지 기본 타입)
 - 다른 자료형으로 분해되지 않는 최소 단위의 데이터 타입
- ▣ 참조형 데이터 타입 (Reference Data Type)
 - 기본 데이터 타입을 제외한 모든 데이터 타입
 - String, 클래스, 배열 등..

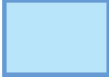
기본형 데이터 타입 (Primitive Data Type)

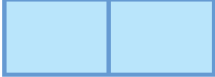
□ 특징

- 기본 타입의 크기는 CPU나 운영체제에 따라 변하지 않음

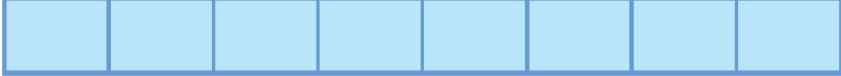
논리 타입 boolean  (1바이트, true 또는 false)

문자 타입 char  (2바이트, Unicode) ('가', '\uac00')

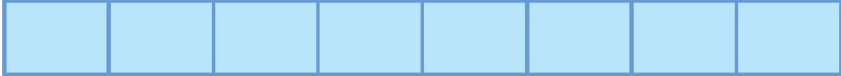
정수 타입 { byte  (1바이트, -128~127)

short  (2바이트, -32768~32767)

int  (4바이트, $-2^{31} \sim 2^{31}-1$) **정수 기본 자료형**

long  (8바이트, $-2^{63} \sim 2^{63}-1$)

실수 타입 { float  (4바이트, $-3.4E38 \sim 3.4E38$)

double  (8바이트, $-1.7E308 \sim 1.7E308$)

실수 기본 자료형

참조형 데이터 타입 (Reference Data Type)

□ String 클래스

- 문자열은 기본 타입이 아님
- String 클래스로 문자열 표현
- 문자열 리터럴 - "JDK", "한글", "계속하세요"

```
String toolName="JDK";
```

- 문자열이 섞인 + 연산 -> 문자열 연결

```
toolName + 1.8          -> "JDK1.8"  
"(" + 3 + "," + 5 + ")" -> "(3,5)"  
System.out.println(toolName + "이 출시됨"); // "JDK1.8이 출시됨" 출력
```

기본 데이터 타입 vs. 참조형 데이터 타입

int age = 10;

age : 10

기본값 저장

String name = "greenjoa";

name : #100

#100 greenjoa

String object

참조값 저장

식별자 (identifier)

- 변수, 상수, 메소드, 클래스 등에 붙이는 이름
- 식별자 작성 규칙
 - 문자, 숫자, '_', '\$'는 사용 가능
 - 자바 9부터는 '_'도 예약어 이므로, 단독으로 이름에 사용하면 오류 (단독 변수명 사용 방지)
 - 자바 언어의 키워드는 식별자로 사용불가
 - 식별자의 첫 번째 문자로 숫자는 사용불가
 - 대소문자 구별
 - `Int barChart`와 `int barchart`는 서로 다른 식별자 선언
 - 보통 변수와 메서드 이름은 소문자로 시작하고, 클래스 이름은 대문자로 시작하는 관례 (Camel Case)
 - `userName`, `getUserName()`, `Person`
 - 상수 변수는 모두 대문자로 작성하며, 단어는 언더스코어(_)로 구분 (Snake Case)
 - `MAX_VALUE`

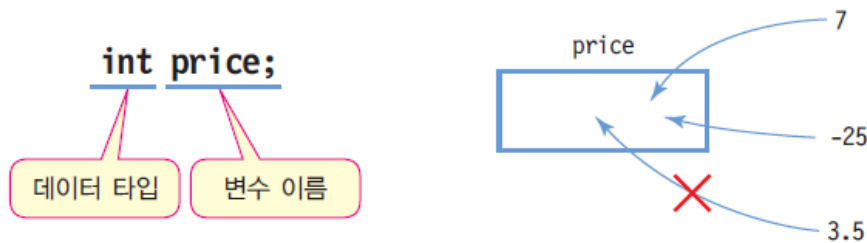
변수 선언

□ 변수

- ▣ 프로그램 실행 중에 값을 임시 저장하기 위한 공간
 - 변수 값은 프로그램 수행 중 변경될 수 있음

□ 변수 선언

- ▣ 데이터 타입에서 정한 크기의 메모리 할당
 - 각 변수에는 해당 타입 값만 저장할 수 있음
- ▣ 메서드 안에서는 어느 위치에나 변수를 선언할 수 있음
 - 사용하기 직전에 변수 선언하는 것을 선호



```
int radius;  
double weight = 75.56;  
char c1 , c2, c3 = 'c';
```

변수 선언 및 초기화

- 선언과 동시에 초기화

- ▣ `int a = 100;`

- 선언과 할당을 분리

- ▣ `int a;`

- ▣ `a = 100;`

- 여러 개를 선언함과 동시에 초기화

- ▣ `int a = 100, b = 200;`

- 변수끼리의 할당

- ▣ `int a = 100;`

- ▣ `int b;`

- ▣ `b = a;`

변수 선언 및 초기화

□ 주의 사항

▣ 메서드 안에 변수를 선언했다면 해당 변수는 반드시 초기화 한 후에 사용해야 함

- 초기화가 되어 있지 않은 지역변수를 사용하면 컴파일 오류가 발생함

```
public class Example {  
    public static void main(String[] args) {  
        int count;  
        count++; // 오류 - 초기화되지 않은 변수를 사용  
    }  
}
```

리터럴과 정수 리터럴

□ 리터럴(literal)

- ▣ 프로그램에서 직접 표현한 값
- ▣ 정수, 실수, 문자, 논리, 문자열 리터럴 있음

□ 정수 리터럴

- ▣ 10진수, 8진수, 16진수, 2진수 리터럴

15	-> 10진수 리터럴 15
015	-> 0으로 시작하면 8진수. 십진수로 13
0x15	-> 0x로 시작하면 16진수. 십진수로 21
0b0101	-> 0b로 시작하면 2진수. 십진수로 5



int n = 15;
int m = 015;
int k = 0x15;
int b = 0b0101;

- ▣ 정수 리터럴은 int 형으로 컴파일
- ▣ long 타입 리터럴은 숫자 뒤에 L 또는 l을 붙여 표시
 - ex) long g = 24L;

실수 리터럴

- ▣ 소수점 형태나 지수 형태로 표현한 실수

- 12. / 12.0 / .1234 / 0.1234 / 1234E-4

- ▣ 실수 타입 리터럴은 double 타입으로 컴파일

```
double d = 0.1234;  
double e = 1234E-4; // 1234E-4 = 1234x10-4이므로 0.1234와 동일
```

- ▣ 숫자 뒤에 f(float)나 d(double)을 명시적으로 붙이기도 함

```
float f = 0.1234f;  
double w = .1234D; // .1234D와 .1234는 동일
```

문자 리터럴

▣ 단일 인용부호(' ')로 문자 표현

- 'a', 'W', '가', '*', '3', '7'

```
char a = 'W';  
char b = '글';  
char c = '\uae00'; // '글'의 유니코드 값(ae00) 사용
```

- \u다음에 4자리 16진수로, 2 바이트의 유니코드(Unicode)

▣ 특수문자 리터럴은 백슬래시(\)로 시작

특수문자 리터럴	의미	특수문자 리터럴	의미
'\b'	백스페이스(backspace)	'\r'	캐리지 리턴(carriage return)
'\t'	탭(tab)	'\"'	이중 인용부호(double quote)
'\n'	라인피드(line feed)	'\''	단일 인용부호(single quote)
'\f'	폼피드(form feed)	'\\'	백슬래시(backslash)

논리 타입 리터럴

□ 논리 값 표시

- true 또는 false 만 사용가능

 - 0과 1은 사용할 수 없음

- boolean 타입 변수에 치환하거나 조건문에 이용

```
boolean a = true;
```

```
boolean b = 10 > 0; // 10>0가 참이므로 b 값은 true
```



```
boolean c = 1; // 타입 불일치 오류. C/C++와 달리 자바에서 1,0을 참, 거짓으로 사용 불가
```

```
while(true) { // 무한 루프
```

```
...
```

```
}
```

Tip: 기본 타입 이외 리터럴

□ null 리터럴

- ▣ 레퍼런스 변수에 대입 사용

 `int n = null;      // 기본 타입에 사용 불가`
`String str = null;`

□ 문자열 리터럴(스트링 리터럴)

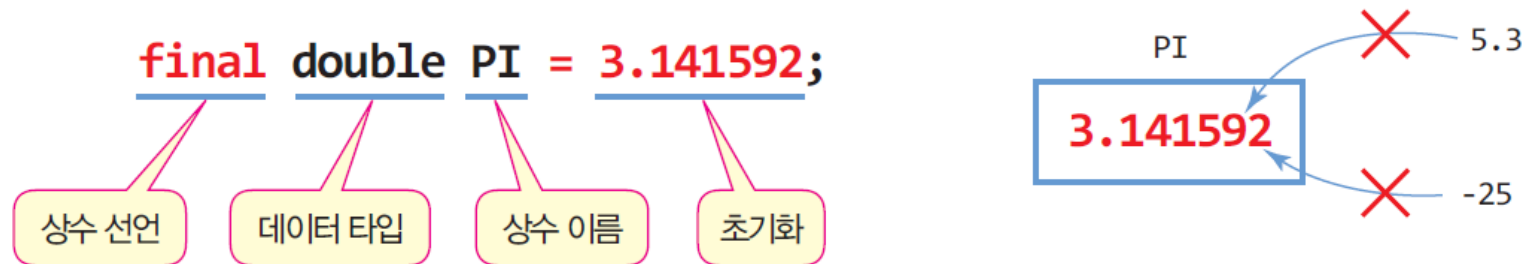
- ▣ 이중 인용부호로 묶어 표현
 - "Good", "Morning", "자바", "3.19", "26", "a"
- ▣ 문자열 리터럴은 String 객체로 자동 처리

```
String str = "Good";
```

상수

□ final 키워드 사용

- 한번 할당하면 변경할 수 없는 값
 - 한번 할당된 후 실행 중 값 변경 불가
- 보통 상수는 대문자로 선언하는 관례



□ final 변수 선언 사례

```
final int LENGTH = 20;
```

```
static final double PI = 3.141592; // static으로 선언하는 것이 바람직
```

상수

□ final 변수 선언 사례

```
final int LENGTH = 20;  
  
final int DAYS_IN_FEB;  
  
If(leapYear){  
    DAYS_IN_FEB = 29;  
}else{  
    DAYS_IN_FEB = 28;  
}
```

```
public class Example {  
    public static final double PI = 3.14;  
    public static void main(String[] args) {  
        }  
}
```

■ System 클래스의 상수

■ System.out

■ public static final PrintStream out;

실습 예제 1. 원의 면적 구하기

- 원의 면적을 구하는 프로그램을 작성하라.
- 코딩 전 결정사항
 - ▣ 원의 면적 구하는 방법
 - 원의면적 = 반지름 * 반지름 * 3.14
 - ▣ 변수 및 타입 결정
 - 반지름, 원주율, 원의면적
 - ▣ 상수와 변수 구분
 - 변수 : 반지름, 원의면적
 - 상수 : 원주율 3.14

실습 예제 1. 원의 면적 구하기 솔루션 예

```
public class CircleArea {  
    public static void main(String[] args) {  
        final double PI = 3.14;           // 원주율을 상수로 선언  
        double radius = 10.2;           // 원의 반지름  
        double circleArea = radius*radius*PI; // 원의 면적 계산  
  
        // 원의 면적을 화면에 출력한다.  
        System.out.print("반지름 " + radius + "-> ");  
        System.out.println("원의 면적 = " + circleArea);  
    }  
}
```

<<실행결과>>

반지름 10.2-> 원의 면적 = 326.68559999999997

타입 변환

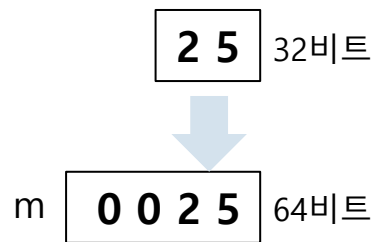
□ 타입 변환

- 한 타입의 값을 다른 타입의 값으로 변환
- 자동 타입 변환과 명시적 타입 변환이 존재
 - 주의) 정수는 `int`, 실수는 `double`이 기본타입
 - 주의) 산술연산자의 결과는 기본 타입을 반환

□ 자동 타입 변환 (Implicit Conversion)

- 컴파일러에 의해 원래의 타입보다 큰 타입으로 자동 변환

```
long m = 25; // 25는 int 타입 25가 long 타입으로 자동 변환  
double d = 3.14 * 10; // 실수 연산 위해 10이 10.0으로 자동 변환
```



타입 변환

□ 명시적 타입 변환 (Explicit Conversion)

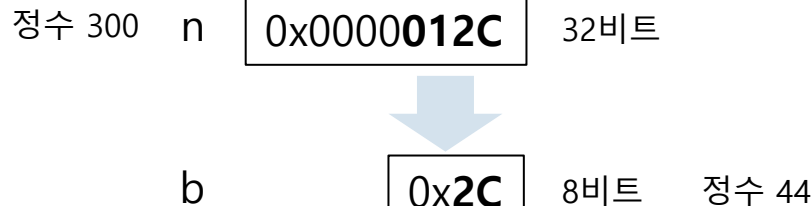
- ▣ 개발자의 의도적 타입 변환
- ▣ () 안에 개발자가 명시적으로 타입 변환 지정

오류 `int n = 300;`
`byte b = n; // int 타입이 byte로 자동 변환 안 됨`

`byte b = (byte)n;` 로 수정

▣ 명시적 타입변환에 의한 강제 변환은 값 손실 우려

`byte b = (byte)n;` 에 따른 손실



`double d = 1.9;`
`int n = (int)d; // n = 1`

강제 타입 변환으로
소숫점 이하 0.9 손실

`short a=1;`
`short b=2;`
`short c = a + b; // 오류 발생 (연산 결과 int)`
`short c = (short)(a + b);`

실습 예제 2. 타입 변환

```
public class TypeConversion {  
    public static void main(String[] args) {  
        byte b = 127;  
        int i = 100;  
        System.out.println(b+i);  
        System.out.println(10/4);  
        System.out.println(10.0/4);  
        System.out.println(((char)0x12340041));  
        System.out.println(((byte)(b+i)));  
        System.out.println(((int)2.9 + 1.8));  
        System.out.println(((int)(2.9 + 1.8)));  
        System.out.println(((int)2.9 + (int)1.8));  
    }  
}
```

227

2

2.5

A

-29

3.8

4

3

0x0041만 읽게 되어 A(65)가 출력

0	0	0	0	0	.	.	.	0	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0	0	0	0	0	.	.	.	0	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0	0	0	0	0	.	.	.	1	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- MSB(most Significant Bit)
 - 가장 왼쪽에 위치한 비트로 부호 결정

요약

□ 자바의 특징

- ▣ 플랫폼에 독립적인 언어
 - 바이트 코드와 자바 가상 기계
- ▣ 객체지향언어
- ▣ 메모리 관리 자동
- ▣ 풍부한 라이브러리 지원

□ 데이터 타입

- ▣ 8개의 기본 데이터 타입
- ▣ 참조형 데이터 타입

□ 변수와 상수

- ▣ 상수는 `final` 키워드 사용하여 선언